DAY 5 — JavaScript Asynchronous Programming

Callbacks • Callback Hell • Promises • then/catch • Chaining • async/await • try/catch

How to study this document

These notes follow the lecture's progression: understand the original problem, try the traditional callback solution, see why it becomes difficult, and then improve it with Promises, .then()/.catch(), and async/await. Each phase explains why the code changes. Examples fill in missing pieces from the pasted code and transcript.

1. Synchronous vs asynchronous execution

Synchronous execution proceeds step by step: the next statement waits for the current operation to finish. Asynchronous execution allows an operation to start while the remaining statements continue. The lecture uses setTimeout() to simulate a slow operation because a normal sum or factorial calculation is too fast to make waiting obvious.

Term	Meaning	Example
Synchronous	The next statement waits for the current operation to finish.	Calculate, then print.
Asynchronous	The operation can continue in the background while later statements run.	setTimeout, network/database operations.
Callback	A function passed to another function so it can be called later; in these examples, after an event/operation.	Function passed to setTimeout.

```
console.log("A");
setTimeout(() => console.log("B – timer finished"), 1500);
console.log("C");
```

Typical output: A, then C, then (after about 1.5 seconds) B. setTimeout starts a timer; it does not block the next statement.

2. Why do we need callbacks?

Lecture idea: asynchronous library functions often provide a callback that tells the program what to do after the operation completes. A dependent statement written on the next line may run before the asynchronous result is ready.

Phase 1 — A tempting but incorrect approach

```
readDataFromDbAsync();
sendResponse();
```

Problem: the response may be sent before the database has returned the data.

Phase 2 — Put the dependent action inside the callback

```
readDataFromDbAsync((data) => {
    sendResponse(data);
});
```

The callback is invoked when the read operation completes, so the response can use the returned data.

Second task: read one table, then write to another

```
readDataFromDbAsync((data) => {
    writeDataInDbAsync(data, () => {
        sendResponse("Read and write completed");
    });
});
```

The write depends on the read, so it goes inside the read callback. The response depends on the write, so it goes inside the write callback. The function names are illustrative, not actual database APIs.

3. Callback nesting and callback hell

When operations are dependent, callbacks can be nested to preserve order. This works, but each additional dependent operation adds another level of nesting.

```
readDataAsync((data) => {
    writeDataAsync(data, () => {
        validateDataAsync(() => {
            sendResponse();
        });
    });
});
```

What suffers?	Why?
Readability	The flow moves deeper to the right and becomes harder to scan.
Maintainability	Changes may require edits across nested blocks.
Debugging	It becomes harder to follow the flow and locate errors.
Scalability	Adding many dependent operations makes the structure tangled.

This deep nesting is called callback hell. Callbacks themselves are not bad; excessive nesting is the problem. The goal stays the same—execute dependent asynchronous operations in the required order—but we need a cleaner technique.

4. Sum → factorial → square: traditional callback phase

Requirement: first calculate sum; then calculate the factorial of that sum; then calculate the square of the factorial. These operations are dependent because each later step needs the previous result.

Phase 1 — All timers at the same level

```
setTimeout(() => console.log("Sum finished"), 1500);
setTimeout(() => console.log("Factorial finished"), 3000);
setTimeout(() => console.log("Square finished"), 2000);
```

Why this fails the requirement: all timers start at roughly the same time. Their completion order is based on their delays, not on the required dependency.

Phase 2 — Nest the callbacks

```
function sum(a, b, callback) {
    setTimeout(() => {
        const result = a + b;
        console.log("Sum is", result);
        callback(result);
    }, 1500);
}

function factorial(num, callback) {
    setTimeout(() => {
        let fact = 1;
        for (let i = 1; i <= num; i++) fact *= i;
        console.log("Factorial is", fact);
        callback(fact);
    }, 3000);
}

function square(num, callback) {
    setTimeout(() => {
        const result = num * num;
        console.log("Square is", result);
        callback(result);
    }, 2000);
}

sum(2, 3, (sumResult) => {
    factorial(sumResult, (factResult) => {
        square(factResult, (squareResult) => {
            console.log("Final result:", squareResult);
        });
    });
});
```

Dry run: sum(2,3) gives 5 after ~1.5 seconds; factorial(5) gives 120 after another ~3 seconds; square(120) gives 14400 after another ~2 seconds. Total is about 6.5 seconds plus small overhead. That waiting is necessary because the operations depend on each other.

Why improve it? The code works, but ten or fifteen dependent operations would create many nested levels.

5. Promises: the next technique

A Promise represents the eventual outcome of an operation. It starts pending, then settles as fulfilled or rejected. Code can respond differently depending on the outcome.

State	Meaning	Action
pending	The operation has not finished yet.	Wait for its outcome.
fulfilled	The operation completed successfully.	Use the result in .then().
rejected	The operation failed or was rejected.	Handle the reason in .catch().

The lecture compares this with a real-life promise: at first, you do not know whether it will be fulfilled. Later, you decide what to do based on the outcome. Similarly, a Promise lets code take different actions depending on success or failure.

Technical detail: the Promise executor runs immediately when new Promise(...) is created. The Promise starts pending. Calling resolve() or reject() settles it; if neither is called, it can remain pending.

6. Create a Promise: new Promise(resolve, reject)

```
const myPromise = new Promise((resolve, reject) => {
    const n = 7;
    if (n % 2 === 0) {
        resolve("Promise fulfilled: number is even");
    } else {
        reject("Promise rejected: number is not even");
    }
});
```

The function passed to new Promise is called the executor function. JavaScript supplies two functions to it: resolve and reject. These are conventional parameter names, not JavaScript keywords. The developer decides which condition means success and which means failure.

Handle the outcome with then() and catch()

```
myPromise
    .then((message) => {
        console.log(message);       // success / fulfilled
    })
    .catch((errorMessage) => {
        console.log(errorMessage);  // rejection
    });
```

With n = 7, the rejection message appears. Change n to 6 to test the fulfilled path. resolve(value) passes a success value to .then(); reject(reason) passes a rejection reason to .catch().

Reusable example: checkEven(num)

```
function checkEven(num) {
    return new Promise((resolve, reject) => {
        if (num % 2 === 0) {
            resolve("Promise fulfilled: number is even");
        } else {
            reject("Promise rejected: number is not even");
        }
    });
}

checkEven(4)
    .then((message) => console.log(message))
    .catch((errorMessage) => console.log(errorMessage));
```

Why make it a function? The same Promise logic can be reused with checkEven(4), checkEven(7), or any other input.

7. Promise chaining: sum → factorial → square

Each operation returns a Promise. Instead of nesting callbacks, return the next Promise from the .then() handler. Returning is essential: it connects the next asynchronous operation to the chain.

```
function sum(a, b) {
    console.log("Started calculating sum...");
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (a > 100 || b > 100) {
                reject("Upper limit crossed. Values must be 100 or less.");
            } else {
                resolve(a + b);
            }
        }, 1500);
    });
}

function factorial(num) {
    console.log("Started calculating factorial...");
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (num < 0 || !Number.isInteger(num) || num > 12) {
                reject("Use an integer from 0 to 12 in this demo.");
                return;
            }
            let fact = 1;
            for (let i = 1; i <= num; i++) fact *= i;
            resolve(fact);
        }, 3000);
    });
}

function square(num) {
    console.log("Started calculating square...");
    return new Promise((resolve) => {
        setTimeout(() => resolve(num * num), 2000);
    });
}

sum(2, 3)
    .then((sumResult) => {
        console.log("Sum is", sumResult);
        return factorial(sumResult);
    })
    .then((factResult) => {
        console.log("Factorial is", factResult);
        return square(factResult);
    })
    .then((squareResult) => {
        console.log("Square is", squareResult);
    })
    .catch((error) => {
        console.log("Operation failed:", error);
    });
```

Read the chain step by step

1. 1. sum(2, 3) returns a Promise. When fulfilled, the first .then() receives 5 as sumResult.
2. 2. return factorial(sumResult) starts factorial(5) and returns its Promise to the chain. The next .then() waits for it.
3. 3. When factorial fulfils, the next .then() receives 120 as factResult.
4. 4. return square(factResult) starts square(120) and returns its Promise.
5. 5. The final .then() receives 14400 as squareResult.
6. 6. If a Promise rejects and the chain does not recover earlier, remaining success handlers are skipped and .catch() handles the rejection.

Common mistake: calling factorial(sumResult) without return. The chain would not wait for that Promise. Return the Promise to connect its outcome to the chain.

8. Why async/await was introduced

Promise chaining removes callback nesting, but a long chain can still contain many .then() calls and return statements. async/await offers a more step-by-step style that is easier to read.

Technique	Advantage	Remember
Nested callbacks	Can enforce dependent order.	Deep nesting can become callback hell.
then/catch	Flatter structure and a shared rejection handler.	Return each dependent Promise.
async/await	Straight-line-looking code with named variables.	Use await in an async function and try/catch for errors.

9. What await and async do

Keyword	Meaning
await	Pauses the surrounding async function until the Promise settles. On fulfilment it evaluates to the result; on rejection it throws the rejection reason.
async	Declares an async function. An async function always returns a Promise.

Important: await does not freeze all JavaScript. It pauses that async function's continuation; unrelated work can continue.

Phase 1 — Why the pasted code gives an error

```
function handlePromise() {
    try {
        var rs = await sum(2, 3); // Error: await is not allowed here
    } catch (error) {
        console.log(error);
    }
}
```

The function signature is missing async. The fix is to declare the function async.

Phase 2 — Correct async/await implementation

```
async function handlePromise() {
    try {
        const sumResult = await sum(2, 3);
        console.log("Sum is", sumResult);

        const factResult = await factorial(sumResult);
        console.log("Factorial is", factResult);

        const squareResult = await square(factResult);
        console.log("Square is", squareResult);
    } catch (error) {
        console.log("Operation failed:", error);
```

```
    }
}

handlePromise();
```

Dry run: sum fulfils with 5; await gives that number to sumResult. Then factorial(5) fulfils with 120; await gives that number to factResult. Then square(120) fulfils with 14400. If any awaited operation rejects, control jumps to catch and the later statements in try are skipped.

10. .catch() versus try/catch

```
/* Promise-chain style */
sum(2, 3)
    .then((value) => factorial(value))
    .then((value) => square(value))
    .then((result) => console.log(result))
    .catch((error) => console.log("Failed:", error));

/* async/await style */
async function run() {
    try {
        const value = await sum(2, 3);
        const fact = await factorial(value);
        const result = await square(fact);
        console.log(result);
    } catch (error) {
        console.log("Failed:", error);
    }
}
run();
```

Both approaches handle a rejection from the operation chain. Do not leave catch empty while learning; log the error or handle it meaningfully.

11. Corrections and important details

Lecture shorthand	More precise understanding
"Callbacks never execute immediately."	In the lecture's async examples, callbacks run later after an event. More generally, a callback is simply a function passed to another function; some APIs can invoke callbacks synchronously.
"Without callbacks, async is impossible."	Useful shorthand for the callback-based APIs discussed in class, but JavaScript also supports Promises, async/await, and event-based APIs.
Promise is resolved or rejected.	Standard states are pending, fulfilled, and rejected. Fulfilled and rejected are settled states; "resolved" is often used more broadly in Promise terminology.
resolve/reject are keywords.	They are parameter names for functions provided to the Promise executor, not reserved keywords.
await makes JavaScript synchronous.	It pauses the current async function's continuation, not the entire JavaScript environment.
A rejection stops the whole program.	It changes the Promise outcome. A chain skips later success handlers until a rejection handler; await throws the rejection reason, which try/catch can handle.
Square was called but not defined in pasted code.	A square() definition is included in these notes so the

	full sum → factorial → square example can run.

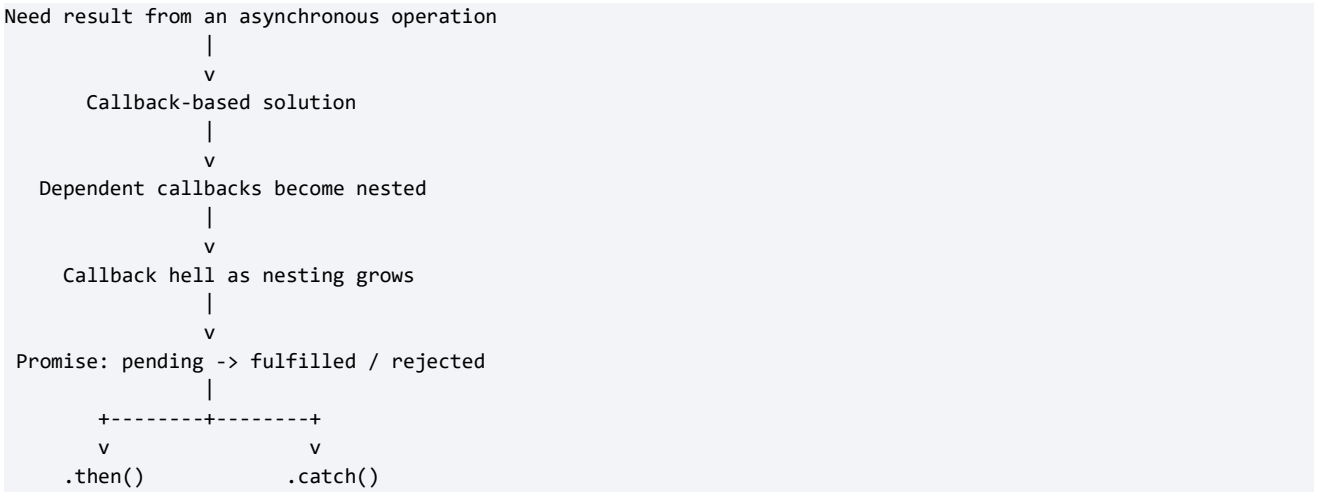
12. Interview-ready definitions

Term	Definition
Callback function	A function passed as an argument to another function so the receiving function can invoke it.
Callback hell	Deeply nested callbacks that make asynchronous code harder to read, maintain, debug, and extend.
Promise	An object representing the eventual success or failure of an asynchronous operation.
resolve(value)	Fulfils a Promise with a value or Promise-like result.
reject(reason)	Rejects a Promise with a reason.
.then()	Registers a fulfilment handler and returns a Promise, enabling chaining.
.catch()	Registers a rejection handler and returns a Promise.
async	Declares an async function, which always returns a Promise.
await	Waits for a Promise's outcome inside an async function, then produces its fulfilment value or throws its rejection reason.

13. Practice tasks — implement these yourself

7. Change checkEven(4) to checkEven(9). Predict the output before running it.
8. Create checkPositive(num): resolve if num > 0; otherwise reject with a useful message.
9. Write multiply(a, b) that returns a Promise and resolves with the product after a 1-second timer.
10. Add a rejection condition to factorial() for values greater than 12. Test factorial(5) and factorial(20).
11. Rewrite sum → factorial → square with async/await and try/catch without looking at the completed solution.
12. Make sum reject intentionally. Explain why factorial and square do not run in the async/await version.
13. Add console.log statements before and after each await, then explain the output order.

14. Final revision map



```
       |
       v
Return each Promise to chain dependent steps
                |
                v
async function + await each Promise + try/catch
```

One-line takeaway: callbacks coordinate what happens after an operation; Promises represent its eventual outcome; .then()/.catch() handle and chain that outcome; async/await is a readable way to write dependent Promise-based steps.